

DATA VORTEX

Round 2 Technical Report

Social Engine NLP: Sentiment Classification and Topic Categorization

Executive Summary

This project develops a text-classification workflow for Social Engine posts, addressing two complementary tasks: sentiment labeling (Negative, Neutral, Positive) and topic categorization (Account_Security, Community_Discussion, Feature_Feedback, Technical_Issues). The archived baseline metrics show 59.2% sentiment accuracy and 90.6% topic accuracy. Team-reported improved results are 61.5% sentiment accuracy and 94.9% topic accuracy, with topic macro F1 rising to 0.72. Rare-topic recall remains an identified limitation.

1. Problem Definition

Social platforms generate large volumes of short, informal text. Manually assigning sentiment and topic labels is labor-intensive and difficult to scale consistently. The project frames this as supervised multi-class text classification: infer sentiment and discussion category from each post's text.

Objectives

- Predict one sentiment label per post.
- Predict one topic category per post.
- Evaluate performance using class-aware metrics, not accuracy alone.
- Produce reproducible artifacts for competition review.

2. Target Audience & Use Case

Potential users include social-platform analysts, community moderators, and product/support teams seeking structured summaries of user feedback. The system can support triage and aggregate analysis; predictions should not be treated as definitive decisions without human review, especially for security-related or low-confidence cases.

3. Proposed Solution & Methodology

The workflow uses labeled post text to fit separate supervised classifiers for sentiment and topic. The improved approach reported by the team combines word-level and character-level TF-IDF features with Logistic Regression and balanced class weights. Word n-grams capture phrase-level signals; character n-grams can capture spelling variants and informal text patterns.

[Insert Figure 1: System workflow — labeled CSV → text preparation → TF-IDF feature extraction (word + char) → Logistic Regression classifiers → metrics, confusion matrices, saved models.]

4. Preprocessing Pipeline

The reported model code applies missing-value handling by filling null post text with an empty string and converting text to strings before fitting/prediction. TF–IDF vectorization is performed within the model pipeline. The word vectorizer uses 1–2-gram features and up to 50,000 features; the character vectorizer uses char_wb 3–5-grams and up to 50,000 features, with sublinear term-frequency scaling.

[Confirm from notebook whether additional normalization, URL/user-handle removal, lowercasing, or duplicate handling was applied; do not claim these unless present in code.]

5. Model Selection & Justification

Logistic Regression is a linear classifier commonly suited to sparse, high-dimensional TF–IDF representations. The improved pipeline uses `class_weight='balanced'` to adjust class contributions in training. A baseline-to-improved comparison was performed, and the team’s validation tuning compared C values 0.5, 1, 2, and 4 using accuracy and macro F1.

For sentiment, C=1.0 produced validation accuracy 0.6757 and macro F1 0.6748, essentially tied with C=2.0 (macro F1 0.6748). These are validation figures; the separately supplied improved evaluation report lists accuracy 0.6149. State the evaluation split explicitly in the final submission.

[Insert Table: full model configurations, baseline algorithm, random seeds, split strategy, and selected C for each task, verified against notebook.]

6. Training Methodology

The project uses labeled examples to train separate sentiment and topic classifiers. The provided tuning code uses a stratified 80/20 validation split with `random_state=42`. A later save-model snippet was provided to fit on `train_df` and serialize models with `joblib`. The ZIP archive contains `sentiment_model.pkl` and `topic_model.pkl`.

[Verify whether the archived pickle files were trained on the full training split or a validation subset, and record the exact train/test partition. Confirm that no held-out test labels were used for tuning.]

7. Results & Evaluation

Task	Baseline accuracy	Improved accuracy	Improved macro F1	Improved weighted F1
Sentiment	0.592	0.615	0.61	0.61
Topic	0.906	0.949	0.72	0.94

The baseline values are from JSON metrics in the project archive; improved values are from the team’s shared classification reports. Because the outputs originate from separately recorded runs, the comparison should be described as reported results unless the notebook confirms identical held-out data and evaluation procedure.

8. Confusion Matrix & Error Analysis

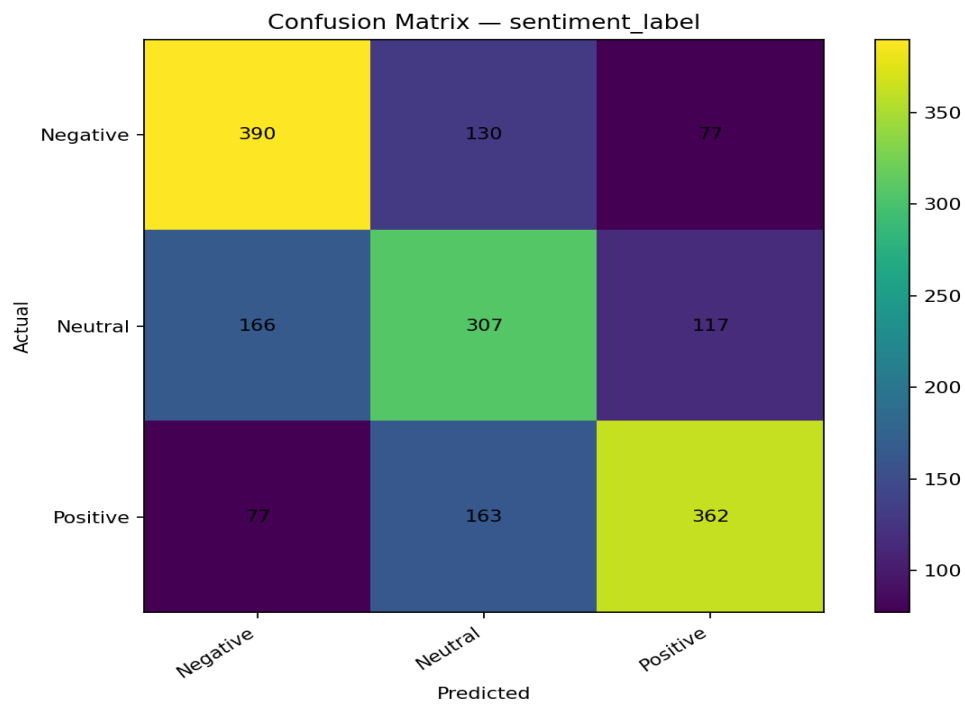


Figure 2. Archived sentiment confusion matrix.

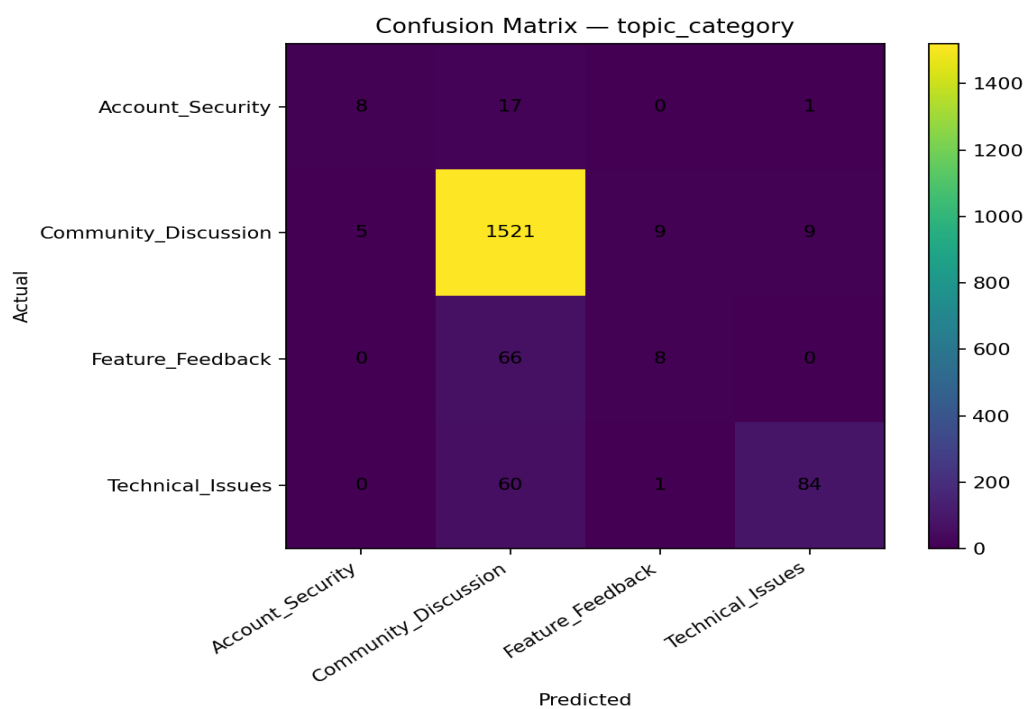


Figure 3. Archived topic confusion matrix.

Sentiment: improved class F1 values are Negative 0.66, Neutral 0.53, and Positive 0.65. Neutral is the lowest-performing class and should be examined for overlap, mixed polarity, and context-dependent phrasing. These are plausible hypotheses; verify with example-level errors.

Topic: Account_Security recall 0.35 and Feature_Feedback recall 0.32 indicate missed positives in the improved report. Their precision values (1.00 and 0.92) suggest predictions for these labels are comparatively selective. Community_Discussion has recall 1.00 and F1 0.97; Technical_Issues has recall 0.85 and F1 0.92.

[Insert 3–5 anonymized representative misclassified examples from topic_error_analysis_worksheet.csv, with true label, predicted label, and concise explanation.]

9. Implementation & Deliverables

The uploaded archive contains NLP_Round2.ipynb, sentiment_model.pkl, topic_model.pkl, sentiment/topic confusion-matrix PNGs, sentiment/topic metrics JSON files, and a topic error-analysis worksheet CSV. These artifacts support code review, model reuse, and evaluation traceability.

10. Limitations & Future Scope

- Improve recall for Account_Security and Feature_Feedback through training-data review, targeted error inspection, and validation-based class-weight or sampling experiments.
- Tune hyperparameters using a dedicated validation split; reserve the final test set for one final evaluation.
- Consider calibrated confidence thresholds and human review for uncertain or sensitive classifications.
- Track per-class precision/recall and macro F1 across future data to detect drift.
- Add reproducibility details: package versions, random seeds, data provenance, and exact split identifiers.

11. Conclusion

The project demonstrates a practical supervised NLP pipeline for two social-text classification tasks. Reported improved performance reaches 61.5% sentiment accuracy and 94.9% topic accuracy, with topic macro F1 of 0.72. Results show strong performance on common topics while revealing continued recall challenges for rare categories. A transparent evaluation protocol and targeted error analysis are central to interpreting these outcomes responsibly.

Submission checklist

- Verify final test metrics and model versions.
- Confirm both confusion matrices correspond to final models.
- Add verified split/preprocessing details and representative error examples.
- Include notebook and trained model files alongside these PDFs.